



# Multi-perspective Concept Drift Detection: Including the Actor Perspective

Eva L. Klijn<sup>(✉)</sup> , Felix Mannhardt , and Dirk Fahland 

Eindhoven University of Technology, Eindhoven, The Netherlands  
{e.l.klijn, f.mannhardt, d.fahland}@tue.nl

**Abstract.** Changes in processes manifest as *concept drift* in event logs. Drift detection aids in analyzing the nature of such change and its impact on the process. Process executions or cases are driven by actors and machines performing the actual work. Actors typically divide and structure their work into *tasks*—multiple consecutive actions performed together—before handing a case to the next actor. Process changes affect this work division and collaboration, potentially impacting performance and outcomes. However, existing research on concept drift detection from event logs has not yet focused on the behavior of actors. We generalize an existing concept drift detection technique to consider actor behavior and control-flow jointly by using a multi-layered event knowledge graph. We evaluate our proposal by comparing the theoretical properties of the newly defined actor perspective features with existing features and perform an experimental evaluation. The experiments showed actor features to be more robust with on average (up to factor 2.6) stronger signals for concept drift in two real-life datasets. Our approach led to new insights into global process changes, changes in behavior of individual actors, and change in collaborations between actors.

**Keywords:** concept drift · multi-perspective · knowledge graph · actors

## 1 Introduction

Organizations rely on insights from process analytics for the successful execution and optimization of their core processes. These processes are driven by human actors and automated resources performing work on the cases of the process. For example, multiple employees of a bank jointly check a credit application, create several loan offers, contact the client for additional information, to finally decline or prepare a contract.

Such processes are subject to change due to changes in process design (top-down) or due to actors gradually changing their way of working (bottom-up) [18]. And both kinds of changes also mutually influence each other and impact process performance and outcomes. Detecting and understanding the nature of changes in a process is therefore essential for continuous process analysis and improvement, such as envisioned in AI-augmented BPM systems [7].

Changes in processes can be detected in event data through *concept drift detection* [4, 19]. Established techniques assume a traditional event log that is structured along the process' control flow and focus on detecting changes in the distribution of

some process' features within a time-window. These features describe in one way or another the control flow logic over all process executions assuming all cases and process steps are impacted equally by a change. However, this leaves bottom-up changes due to individual actors or changes only impacting individual actors undetected.

Yet, actors working on these process instances do exhibit behavior of their own leading to bottom-up changes. They jointly structure and divide work *along a case*, e.g., one actor creates and sends two loan offers to the same client, before handing it to the next actor; such a larger unit of work is called *task* [14, 18] in routines research. Each actor also plans and structures their own work *across multiple cases* [15]. These bottom-up actor-based dynamics in process executions are central for realizing agent-based process management [20] and AI-augmented BPM systems [7]. However, changes in these dynamics are undetectable with drift detection on control-flow features only (detailed in Sect. 2).

In this paper, we investigate detecting changes in the perspective of the actor and control-flow together from event data and how these compare to (1) changes in the control flow only, (2) changes in individual actors and (3) changes in collaborations between actors. As actor behavior is inherently intertwined with control-flow [16], we turn to *event knowledge graphs* (EKGs) [9] as data model (recalled in Sect. 3) to translate an event log into a joint description of actor behavior, process control-flow, and their interactions.

We then address the problem of *defining features for concept drift detection over EKGs that consider the actor and the control-flow perspective jointly*. We generalize existing change point detection techniques from event logs to EKGs (Sect. 4). Such techniques detect drift by extracting multi-variate time-series of the frequency of various process features and then detect at which points the signals described by these time-series significantly change [1]. To detect changes in combined actor and control-flow behavior, we explore and compare several aggregations over EKGs [9] to construct features of multi-variate time series for change point detection. We identified task executions (subsequent actions performed by the same actor on the same case) and abstracted these to task variants (same activity sequence) or tasks (similar task variants) together with the actor and case-paths connecting them. We discuss the features' properties for drift detection in a theoretical analysis.

In our evaluation (Sect. 5), we show for two real-life event logs that including the actor perspective leads to more robust features with on average stronger signal changes, with up to a factors 2.6 stronger in BPIC'17. We also show that it revealed (1) new insights into global process changes (some performance related), (2) changes in individual actors only (some against process change) and (3) task collaborations between actors reducing in favor of actors sequencing tasks themselves. We conclude in Sect. 6.

## 2 Related Work

Concept drift detection and model adaptation has been extensively researched [17]. Change point detection methods [21] based on time series indicate possible drifts by finding a gradual or a sudden change in the observed distributions. We briefly review related work on concept drifts of process behavior [4] as well as the analysis of actor behavior.

A recent survey on concept drift detection in process mining [19] categorizes 45 papers based on their addressed perspectives: control-flow, data and resource. However, despite the importance of actors in organizations, who drive the process behavior, none of the papers covered the resource perspective. An earlier survey [8] also found that only a few approaches supported multiple perspectives, and those were focused on process variant comparison instead of general detection of concept drift. More recently, Adams et al. [1] proposed a detection method for concept drift that does support resources. The idea is to aggregate the information on them in the event log (e.g., workload, active resources) into a numeric feature and, then, use the PELT-algorithm [21] to detect change points. Their approach supports object-centric event logs by extracting individual process executions from an object graph; however, the actor-case interplay is not taken into account at the graph level. Additionally, their evaluation on the BPIC'17 dataset considers only the resource workload as feature. A concept drift with an increased workload for resources is found but none of the additional changes detected by our proposed approach. Thus, while we adopt PELT as change point detection from [1], their approach and proposed feature sets did not find the changes in actor-case interplay or behavior on task level.

Information on actors in processes has been previously used in process mining to analyze how resources organize their work [5, 11], how they work together in collaborations [13], and to build overall organizational models [23]. Also related is a proposal on using agent mining [20] for process analysis. Agent mining puts the actor or agent driving the business process at the center of the investigation and aims to build a multi-agent system by discovering behavior of individual agents. Such agent models could show similar behavior to what we discover as tasks. None of these works investigates concept drift and the particular actor-case interplay feature on a task level that we propose to use. In previous work, we described how to detect and classify task executions by actors in processes [14] and used them to analyze the actor behavior [15]. Both [14] and [15] did not look at changes of such behavior over time in a structured manner.

### 3 Preliminaries

To model behavior of case and actor jointly, we make use of an event knowledge graph (EKG) data model, recalled in Sect. 3.1. Section 3.2 then recalls a way to summarize joint behavior of case and actor in a task instance layer and 3.2 recalls basic aggregation operations for EKGs; we will later use both to derive actor-control-flow features for concept drift detection in Sect. 4.

#### 3.1 Event Knowledge Graphs

A process-aware system can record an action execution as an *event* in an event log. We require that each event records at least the *action* that occurred, the *time* of occurrence, the *case* in which the event occurred and the *resource* (or actor) executing the action.

While an event log orders events wrt. one identifier into isolated event sequences, an *event knowledge graph* (EKG) allows to model that an event is part of multiple, synchronizing paths (e.g., path of a case and path of an actor) [10]. EKGs are based

on *labeled property graphs* (LPG), a graph-based data model supported by graph DB systems [3] that describes concepts as nodes and various relationships between them as edges. In an LPG  $G = (X, Y, \Lambda, \#)$ , each node  $x \in X$  and each relationship  $y \in Y$  with edge  $\vec{y} = (x, x')$  from  $x$  to  $x'$  has a label  $\ell \in \Lambda$ , denoted  $x \in \ell$  or  $y \in \ell$  that describes the *concept* represented by  $x$  or  $y$ ; we use  $(x, x') \in \ell$  as shorthand for  $y \in \ell$ .  $\#_{(a)}(x) = v$  and  $\#_{(a)}(y) = v$  denotes that property  $a$  of  $x$  or  $y$  has value  $v$ ; we use and  $x.a = v$  and  $y.a = v$  as short-hand.

In an EKG, each event and each entity (i.e., each case or resource) is represented by a node with label *Event* and *Entity*, respectively. Each node  $e \in Event$  defines  $e.action$  and  $e.time$ ; each node  $n \in Entity$  defines  $n.type$  and  $n.id$ . While EKGs allow to model arbitrarily many entity types, we subsequently restrict ourselves to EKGs with *two* entity types: *case* (any data object or a classical case identifier) and *resource* (the actors working in the process). Figure 1 (layer 1) shows an example graph: each square node is an *Event* node; each circle is an *Entity* node of the corresponding type (blue for *case*, red for *resource*). An EKG has relationship labels:

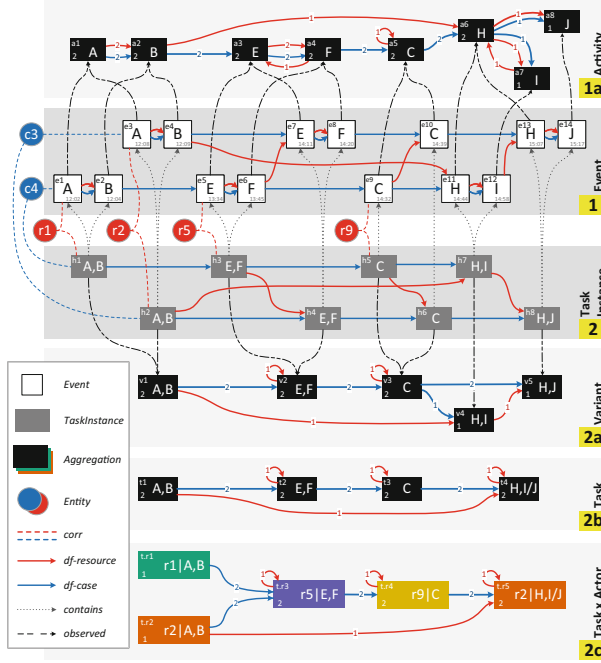
- *corr* (correlation):  $y \in corr, \vec{y} = (e, n)$  iff event  $e \in Event$  is correlated to entity  $n \in Entity$ ; we write  $(e, n) \in corr$  as short-hand.
- *df* (directly-follows):  $y \in df, \vec{y} = (e, e')$  iff events  $e, e'$  are correlated to the same entity  $n$   $(e, n), (e', n) \in corr, e.time < e'.time$  and there is no other event  $(e'', n) \in corr$  with  $e.time < e''.time < e'.time$ ; we write  $(e, e')^{n.type} \in df$  as short-hand, i.e.,  $(e, e')^c$  for entity type *case* and  $(e, e')^r$  for *resource*.

In Fig. 1 (1), *corr* relationships are shown as dashed edges, e.g.,  $e1, e2, e5, e6, e9, e11, e12$  are correlated to case  $c4$  and  $e5, e6, e7, e8$  are correlated to resource  $r5$ . *df*-relationships are shown as solid edges. The *df*-relationships between the events correlated to the same entity form a *df-path* for that entity; the graph in Fig. 1 (1) has 2 *df*-paths for case entities, e.g.,  $\sigma_{c4} = \langle (e1, e2)^c, (e2, e5)^c, (e5, e6)^c, (e6, e9)^c, (e9, e11)^c, (e11, e12)^c \rangle$  and 4 *df*-paths for resource entities, e.g.,  $\sigma_{r5} = \langle (e5, e6)^r, (e6, e7)^r, (e7, e8)^r \rangle$ . See [9] for details on the creation of an EKG  $G$  from event data sources through graph DB queries.

### 3.2 Adding a Task Instance Layer

Events and *df*-edges together form an *event layer* of an EKG [10]. A maximal sub-graph of events  $ti = \{e_1, \dots, e_k\}$  where the *df*-paths of a case and of an actor synchronize is called a task instance [14] and describes the maximal sequence of activities, or unit of work, performed by the same actor in a case. In Fig. 1 (1) sub-graphs of events that meet these criteria are  $\{e_1, e_2\}, \{e_3, e_4\}, \{e_5, e_6\}, \{e_7, e_8\}, \{e_9\}, \{e_{10}\}, \{e_{11}, e_{12}\}$  and  $\{e_{13}, e_{14}\}$ .

To aid analysis, an EKG can be extended with a *task instance layer* [10] by aggregating each task instance sub-graph  $ti = \{e_1, \dots, e_k\}$  into a new “high-level” event node  $h_{ti} \in TaskInstance$ , as shown in Fig. 1 (layer 2);  $h_{ti}$  is connected to each  $e_j \in ti$  via a relationship  $(h_{ti}, e_j) \in contains$ . The *corr* and *df* relationships (for cases and actors) are lifted from *Event* nodes to *TaskInstance* nodes along the *contains* relationships. For example,  $ti1$  over  $e1, e2$  and  $ti4$  over  $e7, e8$  derived



**Fig. 1.** Example EKG containing an event (1) and task instance (2) layer, and aggregation layers for activity (1a), variants (2a), tasks (2b) and tasks per actor (2c). (Color figure online)

from Fig. 1 (1) results in nodes  $h1$  and  $h4$  in Fig. 1 (2), respectively. Further, property  $h_{ti}.variant = e_1.action, \dots, e_k.action$  describing the sequence of actions performed in  $ti$ , and  $h_{ti}.start = e_1.time$  and  $h_{ti}.end = e_k.time$ .

### 3.3 Aggregating Behavior

We recall generalized node and  $df$  aggregation operations on an EKG  $G = (X, Y, \lambda, \#)$  (with event or task instance layers) for process analysis [15].

**Node Aggregation.**  $Agg^N(\alpha, X', l, l')$  aggregates nodes  $x \in X' \subseteq X$  with the same feature  $\alpha(x)$  into concept  $\ell$  as follows: (1) query all values  $V = \{\alpha(x) \mid x \in X'\}$ , (2) for each value  $v \in V$  add a new node  $v \in \ell$  to  $G$  with label  $\ell$ , (3) query the nodes  $X'_v = \{x \in X' \mid \alpha(x) = v\}$  and (4) add new relationship  $y \in \ell', \vec{y} = (x, v)$  with label  $\ell'$  from  $x$  to  $v$ , and (5) set  $v.count = |X'_v|$ . For example, applying  $Agg^N(\alpha_{action}, Event, Activity, observed)$  with  $\alpha_{action}(e) = e.action$  on the graph in Fig. 1 creates one new *Activity* node for each value of the *Event* nodes' *action* property, i.e., nodes  $a1, \dots, a8$  shown in Fig. 1 (layer 1a), and links each event to *Activity* that was observed when the event occurred.

**Directly-Follows Aggregation.**  $Agg^{df}(type, \ell, \ell')$  aggregates (or lifts)  $df$ -relationships (between *Event* or *TaskInstance* nodes) for a particular entity *type* to  $\ell$  nodes along

the  $\ell'$  relationships as follows: (1) for any two nodes  $x, x' \in \ell$  query the set  $df_{x,x'}^{type} = \{(z, z')^{type} \in df \mid (z, x) \in \ell', (z', x') \in \ell'\}$  of all df-edges  $(z, z')$  for  $type$  where nodes  $z, z'$  (e.g., events or task instances) are related to  $x, x'$  via  $\ell'$ , (2) if  $df_{x,x'}^{type} \neq \emptyset$  create a new df-relationship  $y^* \in df, \vec{y^*} = (x, x'), y^*.type = ttype$  and set  $y^*.count = |df_{x,x'}^{type}|$ . For example, first aggregating events to *Activity* nodes via node aggregation, and then applying  $Agg_{df}(type, Activity, observed)$  for  $type \in \{Case, Resource\}$  on the graph in Fig. 1 yields the df-edges between  $a1, \dots, a8$  in Fig. 1 (layer 1a) where, e.g.,  $(a4, a3)^r$  originates from  $(e6, e7)^r$  while  $(a3, a4)^c$  originates from  $(e5, e6)^c$  and  $(e7, e8)^c$ .

Several aggregations using  $Agg^N$  and  $Agg^{df}$ , also limited to task instances in a specific time window, have been proposed [15] as we exploit in our method.

## 4 Method

We generalize an existing change point detection technique (Sect. 4.1) to EKGs (Sect. 4.2). We introduce several features for drift detection in actor behavior and control-flow together (Sect. 4.3) and analyze their theoretical properties for drift detection (Sect. 4.4).

### 4.1 Existing Change Point Detection

We first recall an existing technique [1] for change point detection on event logs structured along the process' control-flow and generalize it to EKGs. Afterwards, we discuss its application to actor features.

Given a log  $L$ , the technique defines a series  $w_0, \dots, w_n$  of subsequent time-windows  $w_i = [t_i; t_{i+1})$  of equal length, e.g., 1 day, (spanning the entire log) and a function  $F$  that extracts from  $L$  per time-window  $w_i$  a feature vector  $\chi_i = F(L, w_i)$ . For example, if  $A = \{a_1, \dots, a_k\}$  are the actions recorded in  $L$  and  $|L_{a_j}, w_i|$  denotes how many events in  $L$  in  $w_i$  refer to action  $a_j$ , then  $F_A(L, w_i) = \langle |L_{a_j}, w_i| / \sum_{b \in A} |L_{b, w_i}| \rangle_{j=1}^k$  defines the feature vector of relative frequencies of activities  $a_1, \dots, a_k$  in  $w_i$ . Likewise a feature vector  $F_A^{df}(L, w_i)$  that defines for each pair  $(a, b) \in A$  how often in a case  $b$  directly follows  $a$  in  $w_i$  can be constructed. Note that df-relations between events in different time windows are ignored.

The feature vectors  $\chi = \langle \chi_1, \dots, \chi_n \rangle$  yield a multi-variate time-series. Change point detection takes  $\chi$  as input and returns as output a series  $0 = cp_0, cp_1, \dots, cp_{k-1}, cp_k = n - 1$  of change points where  $cp_i$  marks a change in  $w_{cp_i} = [t_i; t_{i+1})$ .

### 4.2 Generalizing Change Point Detection to EKGs

Our input for change point detection is an EKG with an event layer and a task instance layer. We generalize the above technique from event logs to EKGs by defining feature extraction functions  $F(G, w_i)$  that extracts features of an EKG  $G$  in a specific time-window. For this, we first aggregate only the nodes in a specific time window  $w_i = [t_i; t_{i+1})$ , and then extract the *count* properties of their aggregates as feature vectors.

Let  $Event|_{w_i} = \{e \in Event \mid t_i \leq e.time < t_{i+1}\}$  be the event nodes in time window  $w_i$ . We write  $Agg(\alpha, Event|_{w_i}, \mathcal{A})$  for first aggregating events in  $w_i$  by  $Agg^N(\alpha, Event|_{w_i}, \mathcal{A}, observed)$  and then lifting  $df$  by  $Agg^{df}(type, \mathcal{A}, observed)$  for  $type \in \{case, resource\}$  as described in Sect. 3.3. For example, choosing  $\alpha_{action}(e) = e.action$ ,  $\mathcal{A} = Activity$  aggregates all  $Event$  nodes in  $w_i$  with  $e.action = a$  to  $a \in Activity$  node and lifts  $df$ .

Let  $G_{w_i}$  be the result of applying  $Agg(\alpha, Event|_{w_i}, \mathcal{A})$  on  $G$ . Then for each node  $a \in \mathcal{A}$ ,  $|G_{a,w_i}^N| = a.count$  defines the *frequency of  $a$  in  $w_i$* , and for  $(a, b) \in \mathcal{A} \times \mathcal{A}$  and  $type$  with  $y \in df$ ,  $\vec{y} = (a, b)$ ,  $y.type = type$ ,  $|G_{(a,b),w_i}^{type}| = y.count$  defines the *frequency of  $b$  directly following  $a$  for a particular entity  $type$  during  $w_i$*  (as the original method, we ignore  $df$ -edges that span multiple windows). The feature vector of relative frequencies of  $\mathcal{A}$  and  $df$  of  $type$  in  $w_i$  are  $F_{\mathcal{A}}^N(G, w_i) = \langle |G_{a,w_i}^N| / \sum_{b \in \mathcal{A}} |G_{b,w_i}^N| \rangle_{a \in \mathcal{A}}$  and  $F_{\mathcal{A}}^{type}(G, w_i) = \langle |G_{(a,b),w_i}^{type}| / \sum_{c,d \in \mathcal{A}} |G_{(c,d),w_i}^{type}| \rangle_{(a,b) \in \mathcal{A} \times \mathcal{A}}$  (for an arbitrary but fixed order of the  $a \in \mathcal{A}$ ). The feature vectors  $F_{\mathcal{A}}^N$  and  $F_{\mathcal{A}}^{type}$  can be used in the existing change point detection technique of Sect. 4.1.

For example, if all events in Fig. 1 (layer 1) occur within one time window  $w = [12; 16)$ , then aggregation yields the “activity layer” of Fig. 1 (layer 1a) describing  $df$ -frequencies for cases and actors, e.g.,  $|G_{(E,F),w}^c| = 2$  and  $|G_{(F,E),w}^r| = 1$ . If we choose  $w_1 = [12; 14)$  and  $w_2 = [14; 16)$ , then  $|G_{(E,F),w_1}^c| = |G_{(E,F),w_2}^c| = 1$  while  $|G_{(F,E),w_i}^r| = 0$  (across windows), i.e., we must choose window length with care.

### 4.3 Extracting Actor and Control-Flow Features

Features derived from the activity layer describe activity counts, case- $df$  counts and even actor- $df$  counts, but do not hold any information on the larger units of work performed, i.e., task executions in which cases and actors synchronize, nor how these are organized by the actors along the cases. To derive features that describe actor and case perspective together, we adopt functions  $F_{\mathcal{A}}(G, w_i)$  to aggregate actor and case behavior in the task instance layer (Sect. 3.2).

In this layer, a task instance—formed by a path of a case and actor together—is the smallest unit describing their interaction or conjoint behavior. Consequently,  $df$ -edges describe handovers from one task to the next: a  $df$ -case edge describes the *handover of a case to another actor* while a  $df$ -resource edge describes the sequencing of tasks *across different cases for the same actor*. To detect drift in the combination of actor behavior and control-flow, we therefore study drift in the frequency of task instances and  $df$ -edges. We can lift the feature extraction on EKG  $G$  from  $Event$  to  $TaskInstance$  nodes: Let  $TI|_{w_i} = \{h_{ti} \in TaskInstance \mid t_i \leq h_{ti}.start \leq h_{ti}.end < t_{i+1}\}$  be the task instance in time window  $w_i$ . Let  $G_{w_i}$  be the result of applying  $Agg(\alpha, TI|_{w_i}, \mathcal{A})$  on  $G$ , i.e., aggregating task instances. Then  $F_{\mathcal{A}}^N(G, w_i)$  and  $F_{\mathcal{A}}^{type}(G, w_i)$  define the feature vectors of relative frequencies of  $\mathcal{A}$  and  $df$  of  $type$  in  $w_i$ .

Previous research [15] identified the following 3 aggregation-levels for *TaskInstances*, which we explore for concept drift detection.

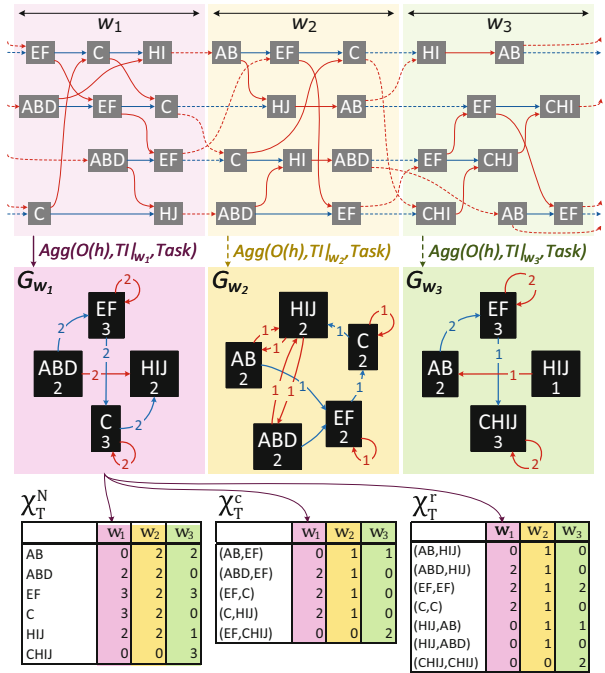
**Task Variant.** Like aggregating events based on the executed action, we can aggregate task instances based on the *sequence of executed actions* (or *variant*). Using  $\alpha(h_{ti}) =$

$h_{ti}$ . *variant* we aggregate into  $\mathcal{A} = \textit{Variant}$  nodes. The resulting *task variant layer* (see layer 2a in Fig. 1) distinguishes each task instance action sequence as a separate feature. Features  $F_{\textit{Variant}}^N$  and  $F_{\textit{Variant}}^C$  (short  $F_V^N$ ,  $F_V^C$ ) describe the relative frequency of each task variant and the handovers from one task variant to another within a case.

**Task.** Aggregating task instances  $h_{ti}$  of *similar variants* into one *Task* requires a “Task oracle” [15]  $O(h_{ti})$  with  $O(h_{ti}) = O(h'_{ti})$  whenever  $h_{ti}.variant$  and  $h'_{ti}.variant$  achieve the same task in similar but different ways, e.g., all variants of creating Offers. A task oracle can be any function to group task variants; this can be done using simple heuristic, e.g., all variants starting with the same activity belong to the same task, or a clustering algorithm. In any case, retrieving a meaningful grouping requires a thorough understanding of the process or domain knowledge. Using  $\alpha(h_{ti}) = O(h_{ti})$  we aggregate into  $\mathcal{A} = Task$  nodes. The resulting *task layer* (see layer 2b in Fig. 1) groups similar variants into tasks resulting in a more granular description of the actor and control-flow perspective. Features  $F_{Task}^N$  and  $F_{Task}^C$  (short  $F_T^N$ ,  $F_T^C$ ) describe the relative frequency of each task and the handovers from one task to another within a case, respectively.

Figure 2 shows how features are extracted from an example task instance layer using  $F_T^p(G, w_i), i \in \{1, 2, 3\}, p \in \{N, c, r\}$  based on the task that was executed according to an oracle  $O$  where CHI and CHJ, and HI and HJ and considered the same task, respectively. For example,  $F_T^N(G, w_1)$  first aggregates task instance nodes in window  $w_1$  based on  $O$  and then extracts feature vector  $\chi_{T,1}^N$  (Fig. 2 left).

**Task Per Actor.** We can also distinguish *Tasks* per *Actor* in the aggregation by using  $\alpha(h_{ti}) = (O(h_{ti}), h_{ti}.actor)$  to aggregate into  $\mathcal{A} = T \times R$  nodes; see layer 2c in Fig. 1. This *task/actor layer* distinguishes tasks for each individual actor separately. It therefore also distinguishes task handovers between a pair of actors, the relative frequency of each task for each actor, and  $F_{T \times R}^T$  the task s



**Fig. 2.** Feature extraction of a task instance layer in  $G'$  for three time windows using function  $F_T(G', w_i), i \in \{1, 2, 3\}$ .

#### 4.4 Properties of Actor and Control-Flow Features

We compare classical control-flow based features, i.e., frequency of (pairs of) activities, to features combining actor and control-flow, i.e., task (variants).

Let  $G_w$  be an EKG with event, task instance, activity, task variant, and task layers derived for window  $w$ . Intuitively, a task variant  $v = \langle a_1, \dots, a_k \rangle$  can only occur as often as the least frequent activity  $a_i$  and the least frequent df-edge  $(a_i, a_{i+1})$  considering both df-case and df-resource edges. Formally, the frequency of task variant  $v = \langle a_1, \dots, a_k \rangle \in \text{Variant}$  in  $G_w$  is  $|G_{v,w}^N| = |\{h \in TI|_w \mid (h, v) \in \text{observed}\}|$ . For each such  $h$  exists a unique set  $e_1, \dots, e_k \in \text{Event}$  s.t.  $(h, e_i) \in \text{contains}$  and  $(e_i, e_{i+1})^c, (e_i, e_{i+1})^r \in \text{df}, i = 1, \dots, k-1$ . Moreover,  $(e_i, a_i) \in \text{observed}, i = 1, \dots, k$  and  $(a_i, a_{i+1})^c, (a_i, a_{i+1})^r \in \text{df}, i = 1, \dots, k-1$ . Thus, the frequency of  $v$  is bounded by the least frequently directly following activities  $(a_i, a_{i+1})^c$  or  $(a_i, a_{i+1})^r$  in the case *or* resource perspective, i.e.,  $|G_{v,w}^N| \leq \min_{i=1}^{k-1} \{|G_{(a_i, a_{i+1}),w}^c|, |G_{(a_i, a_{i+1}),w}^r|\}$ .

This particularly means that every single activity pair  $(a_i, a_{i+1})$  can act as a strong “signal filter”. Suppose  $(a_i, a_{i+1})^c$  is frequent over all time windows but  $a_i$  and  $a_{i+1}$  were performed initially by different actors but from  $w$  on is performed by the same actor, i.e.,  $(a_i, a_{i+1})^r$  was initially infrequent and become frequent from  $w$  on. Then the task variant feature  $v$  changes from infrequent to frequent, i.e., is a notable change at a drift point, while the control-flow feature  $(a_i, a_{i+1})^c$  shows no change.

Conversely, a task  $t$  occurs as often as all its task variants together. Formally, the frequency of  $t \in \text{Task}$  in  $G_w$  is  $|G_{t,w}^N| = |\{h \in TI|_w \mid O(h) = t\}|$ . For each such  $h$  exists a unique variant  $v, (h, v) \in \text{observed}$ . Let  $v_1, \dots, v_k$  be the variants of  $t$ . Thus,  $|G_{t,w}^N| = \sum_{i=1}^k |G_{v_i,w}^N|$  and  $|G_{t,w}^N| \geq |G_{v_i,w}^N|, i = 1, \dots, k$ . Thus, for tasks with many variants that all change similarly in frequency,  $|G_{t,w}^N|$  creates a stronger “change signal”, while for tasks where some variants rise in frequency and some fall in frequency,  $|G_{t,w}^N|$  creates a weaker or no “change signal” compared to  $|G_{v_i,w}^N|$ .

### 5 Experimental Results

We experimentally investigate drift detection of the actor and the control-flow perspective together compared to control-flow based drift detection on two real-life event logs. We implemented the approach of Sect. 4 in Python 3.10<sup>1</sup>; the script (1) generates Cypher queries from [15] for extracting features from the graph, and (2) derives multi-variate time-series and (3) detects the change points based on [1].

#### 5.1 Experimental Setup and Procedure

Our objectives are to:

- O1 Compare change points detected in the entire process using task  $F_T^N$  and task variant  $F_V^N$  features vs the activity  $F_A^N$  and activity-case-df  $F_A^c$  features that were already proposed in [1];

<sup>1</sup> <https://github.com/multi-dimensional-process-mining/ekg-bpic17-concept-drift-detection-multi-perspective>.

- O2 Compare change points detected in individual actors using tasks  $F_{T \times r}^N$  vs change points detected in the entire process using task features  $F_T^N$ ;
- O3 Compare change points detected in individual actors using task-actor-df features  $F_{T \times r}^r$  (i.e., task switching), and between individual actors using task-case-df  $F_{T \times r}^c$  (i.e., case handovers) vs change points detected in the entire process using task features  $F_T^N$ .

For (O1–O3) we compare moment of change and which feature changes in frequency (direction and magnitude). We selected the BPIC’17 data [6] and a private dataset of a small manufacturing process (MF) as these have actor information recorded on the level of detail and quantity required for our approach. For each dataset, we constructed an event knowledge graph [9] including a task instance layer [14] (see Sect. 3), which resulted in 271,815 and 20,776 task instances describing 2,023 and 549 unique variants, respectively. We removed task instances of infrequent task variants (occurring  $< 10$  (1%) and  $< 25$  (4%) times), resulting in 389 and 81 variants, respectively.

To derive the task and task-df features, we used agglomerative clustering of task instances into tasks [2] as task oracle (see Sect. 4): we used Euclidean distance between  $h_{ti,variant}$  as distance metric and chose the number of clusters by maximizing the silhouette index [2], resulting in 21 and 17 clusters for the BPIC’17 and MF data, respectively. We subdivided clusters of leftover variants after manual inspection as follows: (1) each variant occurring at least 100 times was grouped together with all variants including the same set of activities; (2) all remaining variants were removed, resulting in 7 and 3 manually created clusters and removal of 7 and 0 variants, respectively.

We extracted multi-variate time series with window = 1 day (Sect. 4) describing (O1) tasks  $F_T^N$ , task variants  $F_V^N$ , activities  $F_A^N$  and activity-case-df  $F_A^c$  in the entire process from BPIC’17 and MF; (O2) tasks  $F_{T \times r}^N$  in each individual actor  $r$  that executes at least 500 tasks in the entire process, and tasks  $F_T^N$  in the entire process from BPIC’17; (O3) task-actor-df  $F_{T \times r}^r$  in each individual actor  $r$  that executes at least 500 tasks in the entire process, task-case-df  $F_{T \times r}^c$ ,  $r \in R$  for each pair  $R$  of individual actors that share at least 300 handovers, and tasks  $F_T^N$  in the entire process from BPIC’17.

We specifically chose a window size of 1 day for all features (O1–O3) to yield a fine-grained analysis on the level of individual actors that is comparable to the process level. While [1] argues for a window size of 7 days to overcome seasonal variability, we want to be able to detect more fine-grained dynamics and changes on an actor level. Such changes may remain undetected with a coarse granularity level of a week. By using data on actor behavior, our approach can address the issue of seasonal variability caused by weekends/holidays nonetheless by using relative counts for detecting process level drifts (O1) correcting for variable workloads, and, for detecting drift in actors and actor pairs (O2 and O3), we use absolute counts and strip the time series of the days no events were recorded for the actor (O2)/one of the actors (O3). We also tested our approach using a window size of 7 days, but found roughly the same change points.

Extracting features from the graph based on task nodes  $F_T^N$  took on average 1.63 s per time window for BPIC’17 and 0.26 per time window for MF on an Intel Xeon CPU (2x)2.4 GHz machine with 64 GB. We detected change points using the PELT-algorithm [1,21] and varied penalty  $\beta \in \{2, 3, 4\}$  to avoid overfitting.

We investigated changes at a detected change point  $cp_i$  in a feature  $F$  further<sup>2</sup>. We use *signal* for the value of a feature over time. We write  $f \in F$  for one feature of the feature vector, e.g., frequency of activity  $a$ . We then compute  $\mu_{(i,i+1),f}$  as the mean value of  $f$  in all time-windows  $\chi_j, cp_i \leq j < cp_{i+1}$  and  $|\Delta\mu_{cp_i,f}| = |\mu_{(i,i+1),f} - \mu_{(i-1,i),f}|$  as *magnitude of the signal change* at  $cp_i$ ; the signal change in a feature is *stronger* if this value is *greater*. Additionally, the signal of a feature shows a *rising trend* if  $\Delta\mu_{cp_i,f} > 0$  and a *falling trend* otherwise. Measuring the magnitude and direction of each feature’s signal separately allows us to qualitatively compare the changes across specific features, e.g., compare changes in tasks (actor perspective) to changes in activities contained in that task (control-flow perspective). For the average and maximum overall signal change, we write  $\mu(|\Delta\mu_F|)$  and  $\max(|\Delta\mu_F|)$  for the average and maximum of  $|\Delta\mu_{cp_i,f}|$  for all  $cp_i$  and  $f \in F$ , respectively.

## 5.2 Results

**Objective O1.** Table 1 summarizes the change points detected in the entire process using feature sets  $F \in \{F_T^N, F_V^N, F_A^N, F_A^c\}$  and penalty  $\beta \in \{2, 3, 4\}$  for BPIC’17 and MF.

*Change points.* For BPIC’17 at  $\beta = 3$  all detected change points (CPs) largely coincide across all feature sets (maximum time difference in CPs across features  $\leq 4$  days); the first and last detected change points in intervals [9; 11] and [365; 368] mark the start and end of the process. For  $\beta = 2$ , we most notably detect an additional CP in interval [145; 146] in tasks  $F_T^N$ , variants  $F_V^N$  and activities  $F_A^N$  while case-df  $F_A^c$  shows no changes. In contrast,  $F_A^N$  replaces the CP at 198 by two CPs at 183 and 191 (which is not visible in the other features). From  $\beta = 4$  onward, the signals in  $F_A^N$  and  $F_A^c$  are not strong enough anymore to detect changes around [327; 330] and signals in  $F_V^N$  no longer detect changes at the start and end of the process. For MF, we observe in Tab. 1 that at  $\beta = 4$  detected change points precisely coincide across all feature sets. At  $\beta = 3$  an additional change point at 7 is detected in the task features and at  $\beta = 2$  additional change points are detected at 102, 106, 137 for  $F_A^N$  and at 137 for  $F_A^c$ .

*Mean Change Signal.* We investigated the magnitude of the signal changes at the change points of BPIC’17 with  $\beta = 3$  and of MF with  $\beta = 4$  (Table 1). We chose these settings as the change points obtained largely coincide across all features, allowing for the

**Table 1.** Process level change points in days since the start of the log for feature sets ( $F_T^N, F_V^N, F_A^N, F_A^c$ ) and three penalty settings ( $\beta$ ).

|         | $\beta$ | $F_T^N$                       | $F_V^N$              | $F_A^N$                       | $F_A^c$               |
|---------|---------|-------------------------------|----------------------|-------------------------------|-----------------------|
| BPIC’17 | 2       | 11 137 145 200<br>328 366 386 | 9 146 196 330<br>368 | 11 145 183 191<br>327 367 385 | 11 198 329 366<br>384 |
|         | 3       | 11 200 328 366                | 9 196 330 368        | 11 198 327 365                | 11 198 329 366        |
|         | 4       | 11 200 328 366                | 196 330              | 11 198 365                    | 11 199 366            |
|         | MF      |                               |                      |                               |                       |
| MF      | 2       | 7 39 91                       | 7 39 91              | 7 39 91 102<br>106 137        | 7 39 91 137           |
|         | 3       | 7 39 91                       | 39 91                | 39 91                         | 39 91                 |
|         | 4       | 39 91                         | 39 91                | 39 91                         | 39 91                 |
|         |         |                               |                      |                               |                       |

most fair comparison between control-flow and actor-based features. We found for BPIC’17 and MF the average signal change  $\mu(|\Delta\mu_F|)$  is stronger in tasks  $F_T^N$  than in other features  $F_V^N/F_A^N/F_A^c$  (1.2 vs 0.2/0.8/0.1 for BPIC’17 and 1.5 vs 0.7/0.7/0.2 for

<sup>2</sup> Full BPIC’17 results available: <https://zenodo.org/doi/10.5281/zenodo.10933096>.

MF). For BPIC'17 the maximum signal change  $\max(|\Delta\mu_F|)$  is significantly stronger in  $F_T^N/F_V^N$  vs  $F_A^N/F_A^c$  (17.4/17.3 vs 6.5/6.7), whereas for MF they were slightly stronger in  $F_A^N/F_A^c$  vs  $F_T^N/F_V^N$  (5.2/6.0 vs 4.0/4.0). Overall, using task features generally led to equal or stronger signal changes, while using activity/activity-df features in most cases led to weaker signal changes confirming the analysis in Sect. 4.4.

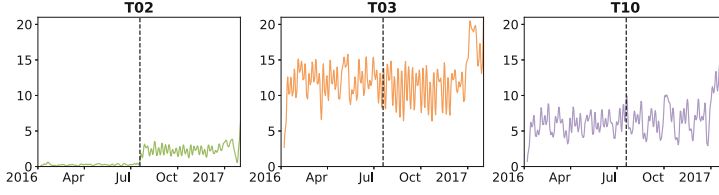
*Comparison of Change Signals Across Features.* On the four CPs ( $cp_1, \dots, cp_4$ ) of BPIC'17 for  $\beta = 3$  shown in Table 1, we investigated how the magnitudes and directions of signal changes compare across  $F_T^N, F_V^N$  and  $F_A^N, F_A^c$ . We again use  $\beta = 3$  to allow for the best comparison as for this  $\beta$  the obtained change points largely coincide for all feature sets. We include in Table 2 for  $cp_2$  and  $cp_3$  all task changes with  $|\Delta\mu_{cp_i, f \in F_T^N}| = |M| > 1.0$  and limit  $cp_1$  and  $cp_4$  to task changes of  $|M| > 10$  due to extensive drift at the process start and end. We include variants of a task if these showed oppos-

**Table 2.** Comparison of change signals in  $F_T^N, F_V^N$  vs  $F_A^N, F_A^c$  at four change points (CP) detected with  $\beta = 3$ .

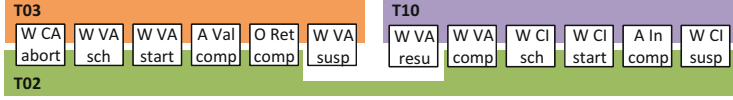
| CP     | $F_T^N$ | $F_V^N$ | $M$           | $F_A^N$    |              |                |              | $F_A^c$    |        |                |              |
|--------|---------|---------|---------------|------------|--------------|----------------|--------------|------------|--------|----------------|--------------|
|        |         |         |               | Same trend |              | Opposite trend |              | Same trend |        | Opposite trend |              |
|        |         |         |               | $\varphi$  | $\max$       | $\varphi$      | $\max$       | $\varphi$  | $\max$ | $\varphi$      | $\max$       |
| $cp_1$ | m04     |         | <b>-11.19</b> | 1.00       | -2.94        | <b>0.00</b>    |              | 0.85       | -1.57  | <b>0.00</b>    |              |
|        | T03     |         | <b>10.13</b>  | 0.97       | 4.65         | <b>0.00</b>    |              | 0.91       | 2.71   | <b>0.00</b>    |              |
|        | T07     |         | <b>-14.71</b> | 0.95       | -3.15        | <b>0.00</b>    |              | 0.94       | -2.32  | <b>0.00</b>    |              |
| $cp_2$ | m01     |         | <b>3.79</b>   | 1.00       | 1.26         | <b>0.00</b>    |              | 0.75       | 0.88   | <b>0.00</b>    |              |
|        | m03     |         | <b>-5.41</b>  | 0.97       | -2.34        | <b>0.03</b>    | <b>0.33</b>  | 0.40       | -2.12  | <b>0.00</b>    |              |
|        | m04     |         | <b>-2.06</b>  | 1.00       | -0.73        | <b>0.00</b>    |              | 0.85       | -0.59  | <b>0.00</b>    |              |
|        | T01     |         | <b>1.37</b>   | 0.72       | 0.90         | <b>0.21</b>    | <b>-2.34</b> | 0.42       | 0.31   | <b>0.21</b>    | <b>-2.12</b> |
|        | T02     |         | <b>2.18</b>   | 0.77       | 1.26         | <b>0.13</b>    | <b>-2.34</b> | 0.72       | 0.64   | <b>0.03</b>    | <b>-0.49</b> |
|        | T03     | v04     | -1.13         | 0.17       | <b>-1.67</b> | <b>0.67</b>    | <b>0.38</b>  | 0.20       | -0.31  | <b>0.60</b>    | <b>0.49</b>  |
|        |         | v06     | <b>1.18</b>   | 0.80       | 0.58         | <b>0.20</b>    | <b>-1.67</b> | 1.00       | 0.49   | <b>0.00</b>    |              |
|        | T10     | v08     | <b>2.40</b>   | 1.00       | 1.26         | <b>0.00</b>    |              | 1.00       | 0.64   | <b>0.00</b>    |              |
|        |         | v13     | -1.61         | 0.33       | <b>-2.34</b> | <b>0.67</b>    | <b>1.26</b>  | 0.20       | -0.49  | <b>0.60</b>    | <b>0.64</b>  |
|        | T11     |         | <b>1.28</b>   | 0.85       | 1.26         | <b>0.03</b>    | <b>-2.34</b> | 0.35       | 0.88   | <b>0.00</b>    |              |
| $cp_3$ | m02     |         | <b>-13.02</b> | 1.00       | -3.17        | <b>0.00</b>    |              | 0.50       | -3.60  | <b>0.00</b>    |              |
|        | m04     |         | <b>1.38</b>   | 0.88       | 0.55         | <b>0.00</b>    |              | 0.85       | 0.65   | <b>0.00</b>    |              |
|        | m05     |         | <b>15.27</b>  | 1.00       | 3.27         | <b>0.00</b>    |              | 1.00       | 4.78   | <b>0.00</b>    |              |
|        | T03     | v04     | <b>-4.79</b>  | 0.00       |              | <b>0.33</b>    | <b>1.67</b>  | 1.00       | -1.57  | <b>0.00</b>    |              |
|        |         | v27     | <b>4.10</b>   | 0.33       | 1.62         | <b>0.00</b>    |              | 0.20       | 1.36   | <b>0.80</b>    | <b>-0.22</b> |
|        | T11     | v12     | <b>-2.14</b>  | 0.00       |              | <b>0.33</b>    | <b>1.67</b>  | 0.50       | -0.58  | <b>0.50</b>    | <b>0.11</b>  |
|        |         | v43     | <b>1.90</b>   | 0.33       | 1.62         | <b>0.00</b>    |              | 1.00       | 0.67   | <b>0.00</b>    |              |
|        | m05     |         | <b>-11.88</b> | 1.00       | -4.23        | <b>0.00</b>    |              | 1.00       | -3.70  | <b>0.00</b>    |              |
| $cp_4$ | T11     |         | <b>17.38</b>  | 0.86       | 6.50         | <b>0.11</b>    | <b>-2.94</b> | 0.62       | 6.72   | <b>0.06</b>    | <b>-0.66</b> |

ing directions for  $\Delta\mu_{cp_i, f \in F_V^N}$  (rising vs falling). For each task or variant change, we report the fraction  $\varphi$  of activities and of activity pairs contained in the task or variant whose signal change  $\Delta\mu_{cp_i, f \in F, F \in F_A^N, F_A^c}$  has the same trend as the signal change for  $F_T^N/F_V^N$ , and the fraction that has the opposite trend (only considering signal changes  $> 0.1$ ). For each, we report the strongest signal change (max).

In Table 2 we observe that only for few task (variants) all the contained activity and activity-df features change in the same trend ( $\varphi = 1.0$ ), and that for 11/22 task (variants) at least some activity and activity-df features (up to 67%) show an opposite trend. In terms of magnitude, we observe that the signal changes in all tasks in  $F_T^N$  are stronger than the max signal changes measured in  $F_A^N, F_A^c$  going the same direction. We find that a few changes in tasks can to some extent be explained by changes in some  $F_A^N$  or  $F_A^c$ , e.g., the change  $-5.41$  in m03 at  $cp_2$  is also detected in 97% of its activities with strongest change  $-2.34$  in “W\_VA+resume”. However, signals in those same activities can show conflicting changes wrt. other task changes, e.g., activity “W\_VA+resume” is also contained in T01 which changes with  $+1.37$  in  $cp_2$ .



**Fig. 3.** Signals of T02, T03 and T10 in  $F_T^N$ .



**Fig. 4.** Most dominant variants of T02 combining T03,T10.

*Variants.* Signal changes in variants in  $F_V^N$  are stronger than the max signal changes in  $F_A^N, F_A^c$  for 6/8 variants. For v04 and v12 at  $cp_3$  no activity ( $F_A^N$ ) changed in the same direction as the variants ( $\varphi = 0$ ). Only v04 and v13 at  $cp_2$  contained activities ( $F_A^N$ ) that have a stronger max signal change, but this only holds for a minority of activities while the majority of contained activities ( $F_A^N$ ) and activity-df ( $F_A^c$ ) shows the opposite trend ( $\varphi \geq 0.6$ ).

*Opposing Variants.* We also observe some strong (opposite) changes in variants that were not reflected in the tasks. For example, task T03 with  $|M| < 1.0$  at  $cp_3$  contains two variants v04 and v27 showing strong opposite changes,  $-4.79$  and  $+4.10$ , respectively, which is also reflected in the max signal change in corresponding activity-df signals ( $F_A^c$ ). Inspection of the variants showed that the activity “W\_CAO+abort” as part of variant v04 was replaced by “W\_CAO+withdraw”.

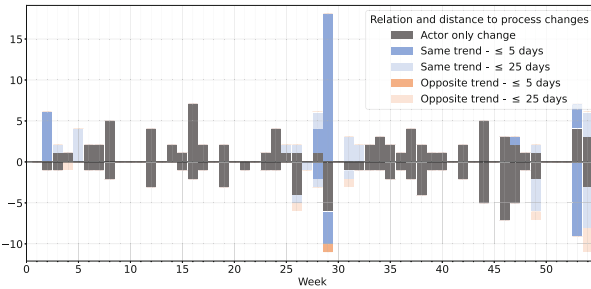
*Impact of Drift on Performance.* More closely inspecting signals and contents of changing tasks in Table 2 revealed task T02 was only introduced as a regular task from  $cp_2$ , see Fig. 3. Even though T02 combines actions of existing tasks T03 and T10, illustrated by their most dominant variants in Fig. 4, the relative increase in T02 did not coincide with (significant,  $|M| < 1.0$ ) relative decreases in T03 and T10, see Fig. 3. We investigated the effect of combining T03 and T10 into T02 after  $cp_2$  on the processing time. Table 3 compares duration of trace suffixes  $\langle X, \dots \rangle$  with  $X = \text{T03, T10}$  vs  $X = \text{T02}$  occurring  $> 50$  times. We observe in Table 3 that the change to T02 at  $cp_2$  led to significantly shorter processing times in 4 out of 7 suffixes.

**Objective O2.** For this objective and the next (O3), we only report the changes in actors/actor pairs detected with  $\beta = 4$ , as these included also the more subtle changes that remained undetected with larger  $\beta$ .

**Table 3.** Average duration of trace suffixes  $\langle X, \dots \rangle$  with  $X = T03, T10$  vs  $X = T02$  occurring  $> 50$  times after  $cp_2$ .

| Suffix                          | X = T02      |     | X = T03, T10 |     | P-value         |
|---------------------------------|--------------|-----|--------------|-----|-----------------|
|                                 | $\mu$        | #   | $\mu$        | #   |                 |
| X, m01, T03, T01                | <b>4d03h</b> | 242 | 4d19h        | 526 | <b>0.012</b>    |
| X, m01, T03, T10, m01, T01      | <b>7d20h</b> | 134 | 12d04h       | 145 | <b>6.950e-7</b> |
| X, m01, T03, T15                | <b>4d07h</b> | 155 | 5d16h        | 58  | <b>0.001</b>    |
| X, m01, T03, m03, T01           | 4d18h        | 63  | <b>4d10h</b> | 113 | 0.463           |
| X, T03, T01                     | <b>3d11h</b> | 33  | 3d17h        | 147 | 0.622           |
| X, m01, T03, T10, m01, T03, T01 | <b>6d14h</b> | 74  | 8d14h        | 39  | <b>0.049</b>    |
| X, m01, m01, T03, T15           | <b>6d22h</b> | 58  | 8d19h        | 18  | 0.098           |

Fig. 5 summarizes the task changes detected in individual actors  $F_{T \times r}^N$  with  $\beta = 4$  compared to the task changes detected in the entire process; it displays 225 task changes in 65 actors. We observe a number of changes peaking around process change points; most changes follow the same trend and largely coincide with the process change, e.g. 27 task changes in actors within 5 days of  $cp_2$  (weeks 28 and 29). We also observe 8 task changes in actors going against the trend of process changes within 25 days of the change point, one of which within 5 days (week 29). Surprisingly, we also see many task changes in individual actors only, i.e., either not happening in proximity of the process change or not reflected in process task changes.

**Fig. 5.** Number of tasks rising (+) and falling (-) in  $F_{T \times r}^N$  in individual actors with  $\beta = 4$  compared to task changes in the entire process  $F_T^N$  ( $\beta = 3$ ).

**Objective O3.** Table 4 summarizes the changes detected in task handovers between actors ( $F_{T \times r}^c$ ) (left) and in task switching in individual actors ( $F_{T \times r}^r$ ) (right) happening within 5 days of process change point  $cp_2$  with  $\beta = 4$ . In Table 4 (left), we

**Table 4.** Number and trend of detected changes in case handovers between individual actors  $F_{T \times r}^c$  (left) and in task switches in individual actors  $F_{T \times r}^r$  (right) within 5 days of  $cp_2$  with  $\beta = 4$ .

| Case handovers $F_{T \times r}^c$ , $c = (t_i, t_j)$ |       |           | Task switches $F_{T \times r}^r$ , $r = (t_i, t_j)$ |       |       |     |       |     |     |
|------------------------------------------------------|-------|-----------|-----------------------------------------------------|-------|-------|-----|-------|-----|-----|
| $a_j$                                                | $a_i$ |           | $a_j$                                               | $a_i$ |       |     |       |     |     |
|                                                      | m03   | T01...T15 |                                                     | m03   | T01   | T02 | T03   | T10 | T15 |
| m03                                                  | -7    |           | m03                                                 | -5    | -2    |     |       | -3  | -3  |
| T01                                                  | -5    |           | T01                                                 | -3    | +5    |     | +2 -2 | +5  | +4  |
| T02                                                  |       |           | T02                                                 |       |       | +6  | +6    |     |     |
| T03                                                  |       |           | T03                                                 |       | +2 -1 | +6  | -3    |     |     |
| T10                                                  | -6    |           | T10                                                 | -3    | +5    |     |       | +4  | +5  |
| T15                                                  | -5    |           | T15                                                 | -1    | +4    |     |       | +4  | +3  |

observe for 5–7 actors a reduction in handovers from m03 to m03, T01, T10 and T15 and observe similar reductions in task switching also between m03 and m03,...,T15; this decrease in m03 is also reflected in the process changes (see Table 2).

Conversely, we observe for 2–5 actors increases in task switching involving T01, T10 and T15, suggesting that handing over these tasks to other actors has been replaced by actors sequencing these tasks more themselves. In Fig. 4 (right), we also observe for 6 actors a uniform increase in task switches involving T02 (in line with the process change) and a non-uniform change in task-switching involving T03.

## 6 Conclusion

We generalized control-flow drift detection to multi-perspective drift detection by feature extraction from a multi-layered event knowledge graph (EKG). We use aggregation in an EKG with actor and case paths to extract novel features about (changes in) actor behavior along the control-flow in the form of task (variants) and handovers between actors. We rely on an existing change point detection [1] but propose an entirely new set of features that is not available in a regular event log.

The experimental evaluation showed that including features combining the actor and the control-flow perspective leads to more robust drift point detection with on average (up to  $2.6\times$ ) stronger signal changes compared to an existing approach using the same detection method. The robustness of task features in drift detection arises because a task sequentially composes multiple control-flow features, thus suppressing “noisy” drift signals of individual control-flow features within the larger task, which we analyzed theoretically and confirmed experimentally. Interestingly, we observed (1) drift on the task level to be correlated with process performance, and (2) individual actors also showing drift independent of control-flow drift in various forms. This shows that actor behavior can play a significant role in real-life processes and suggests that detecting drift in the actor perspective is essential for continuous process analysis and improvement [7] and can provides new insights in the context of (changing) routines [16, 18], effect of process redesign [22] or fraud detection in auditing [12]. Further research on the impact of the relationship between resource and control-flow perspective on process performance could help with the identification of root causes for performance deviations.

Our findings rely on the quality of the task clustering and to what extent clustering yields the real-life tasks in the process; doing this properly requires a detailed understanding of the process. Furthermore, the task clustering in itself still requires manual support, hindering the integration into a continuous monitoring system. A limitation shared with all other drift detection approaches is that our approach ignores edges spanning a time window, potentially leaving certain drifts undetected. An EKG based approach could consider them but this requires further research. A next step is to investigate a more systematic approach to analyze cause-effect relations in more detail, possibly including synthetic datasets and a more comprehensive parameter exploration to systematically evaluate the detection method. Moreover, our feature extraction on EKG is generic, enabling further research on drift detection in other process dimensions such as data objects, relations, or queues [10]. Finally, whereas we showed feasibility on

two real-life datasets, the scalability of the approach still needs to be systematically investigated.

## References

1. Adams, J.N., van Zelst, S.J., Rose, T., van der Aalst, W.M.P.: Explainable concept drift in process mining. *Inf. Syst.* **114**, 102177 (2023)
2. Aggarwal, C.C.: *Data Mining - The Textbook*. Springer, Cham (2015). <https://doi.org/10.1007/978-3-319-14142-8>
3. Bonifati, A., Fletcher, G.H.L., Voigt, H., Yakovets, N.: *Querying Graphs. Synthesis Lectures on Data Management*. Morgan & Claypool Publishers (2018)
4. Bose, R.P.J.C., van der Aalst, W.M.P., Zliobaite, I., Pechenizkiy, M.: Dealing with concept drifts in process mining. *IEEE Trans. Neural Netw. Learn. Syst.* **25**(1), 154–171 (2014)
5. Delcoucq, L., Lecron, F., Fortemps, P., van der Aalst, W.M.P.: Resource-centric process mining: clustering using local process models. In: *SAC 2020*, pp. 45–52. ACM (2020)
6. van Dongen, B.F.: *BPI challenge 2017. Dataset* (2017). <https://doi.org/10.4121/12705737.v2>
7. Dumas, M., et al.: AI-augmented business process management systems: a research manifesto. *ACM Trans. Manag. Inf. Syst.* **14**(1), 11:1–11:19 (2023)
8. El-Khawaga, G., Abu-Elkheir, M., Barakat, S.I., Riad, A.M., Reichert, M.: CONDA-PM - a systematic review and framework for concept drift analysis in process mining. *Algorithms* **13**(7), 161 (2020)
9. Esser, S., Fahland, D.: Multi-dimensional event data in graph databases. *J. Data Semant.* **10**, 109–141 (2021)
10. Fahland, D.: Process mining over multiple behavioral dimensions with event knowledge graphs. In: van der Aalst, W.M.P., Carmona, J. (eds.) *Process Mining Handbook*. LNBIP, vol. 448, pp. 274–319. Springer, Cham (2022). [https://doi.org/10.1007/978-3-031-08848-3\\_9](https://doi.org/10.1007/978-3-031-08848-3_9)
11. van Hulzen, G.A.W.M., Li, C., Martin, N., van Zelst, S.J., Depaire, B.: Mining context-aware resource profiles in the presence of multitasking. *Artif. Intell. Med.* **134**, 102434 (2022)
12. Jans, M., Eulerich, M.: Process mining for financial auditing. In: van der Aalst, W.M.P., Carmona, J. (eds.) *Process Mining Handbook*. LNBIP, vol. 448, pp. 445–467. Springer, Cham (2022). [https://doi.org/10.1007/978-3-031-08848-3\\_15](https://doi.org/10.1007/978-3-031-08848-3_15)
13. Jooen, L., Jans, M., Depaire, B.: Mining valuable collaborations from event data using the recency-frequency-monetary principle. In: *CAiSE 2022*. LNCS, vol. 13295, pp. 339–354. Springer, Cham (2022). [https://doi.org/10.1007/978-3-031-07472-1\\_20](https://doi.org/10.1007/978-3-031-07472-1_20)
14. Klijn, E.L., Mannhardt, F., Fahland, D.: Classifying and detecting task executions and routines in processes using event graphs. In: *BPM 2021*. LNBIP, vol. 427, pp. 212–229. Springer, Cham (2021). [https://doi.org/10.1007/978-3-030-85440-9\\_13](https://doi.org/10.1007/978-3-030-85440-9_13)
15. Klijn, E.L., Mannhardt, F., Fahland, D.: Aggregating event knowledge graphs for task analysis. In: Montali, M., Senderovich, A., Weidlich, M. (eds.) *ICPM 2022*. LNBIP, vol. 468, pp. 493–505. Springer, Cham (2022). [https://doi.org/10.1007/978-3-031-27815-0\\_36](https://doi.org/10.1007/978-3-031-27815-0_36)
16. Kremser, W., Blagoev, B.: The dynamics of prioritizing: how actors temporally pattern complex role-routine ecologies. *Adm. Sci. Q.* **66**(2), 339–379 (2021)
17. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under concept drift: a review. *IEEE Trans. Knowl. Data Eng.* **31**(12), 2346–2363 (2018)
18. Pentland, B., Feldman, M., Becker, M., Liu, P.: Dynamics of organizational routines: a generative model. *J. Manag. Stud.* **49**, 1484–1508 (2012)
19. Sato, D.M.V., Freitas, S.C.D., Barddal, J.P., Scalabrin, E.E.: A survey on concept drift in process mining. *ACM Comput. Surv.* **54**(9), 189:1–189:38 (2022)

20. Tour, A., Polyvyanyy, A., Kalenkova, A.A.: Agent system mining: vision, benefits, and challenges. *IEEE Access* **9**, 99480–99494 (2021)
21. Wambui, G.D., Waititu, G.A., Wanjoya, A.K.: The power of the pruned exact linear time (PELT) test in multiple changepoint detection. *AJTAS* **4**, 581–586 (2015)
22. Wurm, B., Grisold, T., Mendling, J., vom Brocke, J.: Business process management and routine dynamics, pp. 513–524. Cambridge University Press (2021)
23. Yang, J., Ouyang, C., van der Aalst, W.M.P., ter Hofstede, A.H.M., Yu, Y.: OrdinoR: a framework for discovering, evaluating, and analyzing organizational models using event logs. *Decis. Support Syst.* **158**, 113771 (2022)